

Disclaimer: Dieses OER-Dokument stammt aus dem Projekt [proTirone](#), das über GitHub [freie Unterrichtsmaterialien](#) samt [Quellcode](#) offeriert. proTirone-Materialien werden unter den Bedingungen der [CC-BY-4.0-Lizenz](#) so angeboten, wie sie sind, ohne Zusage bestimmter Eigenschaften und ohne Gewährleistung (§5). Dafür dürfen sie – bei angemessener Namensnennung (§3) – für beliebige (auch kommerzielle) Zwecke verändert und weitergegeben werden (§2).

LF09:16:Ver- und Entschlüsselung

1. Einstieg [→ ZP:Sheet2]

ÜBUNG LF09:16:Verschlüsselung:01

- ☐ Bitte entschlüsseln Sie den Text

```
1 xvd-su!ydwH-gdwhq-yrq-vlfkhukhlwvsrolwlnhuq-lp-qhwC
```

Lösung:

- Der Text steht für `usa-private-daten-von-sicherheitspolitikern-im-netz`
- Er ist Teil eines Links zu einem Spiegelartikel:

<https://www.spiegel.de/netzwelt/web/usa-private-daten-von-sicherheitspolitikern-im-netz-a-132886fa-7d7e-4698-9507-a3423b23bddc>

- Die Verschlüsselung ist die Caesar-Verschlüsselung in der Standardkonfiguration
 - Algorithmus: Verschiebe den realen Buchstaben um n Buchstaben nach rechts.
 - Schlüssel: n
- → Onlinetool: <https://www.kryptowissen.de/caesar-chiffre-praxis.php>
- → Algorithmus: <https://www.kryptowissen.de/caesar-chiffre.html>
-

→ Veranschaulichung:

<https://de.wikipedia.org/wiki/Caesar-Verschl%C3%BCsslung>

2. Begriffe [→ ZP:Sheet3]

- **Verschlüsselung** (= Chiffrierung, Codierung)

- = Umwandlung (elektronischer) Texte, Bilder, Sprachnachrichten, Sourcecode, Payloads eines Netzpaketes - will sagen: den 'Klartext' - "[...] in eine unverständliche Zeichenfolge" - will sagen: in den 'Geheimtext' (Chiffre [BSI])
- dient: der Geheimhaltung und soll "[...] gegen unbefugtes 'Mitlesen' zu schützen".
- braucht: i.d.R einen Schlüssel und eine Vorschrift, wie der auf den Klartext anzuwenden ist, damit das Chiffre entsteht.
- **Entschlüsselung** (= Dechiffrierung, Decodierung)
 - meint die systematische Umkehr der Verschlüsselung in Kenntnis des Schlüssels.
- **Entzifferung**
 - meint die 'Enträtselung' eines codierten Textes in Unkenntnis Schlüssels
- **Kryptologie** = Wissenschaft von der Verschlüsselung (Oberbegriff)
 - *Kryptanalyse* (= Kryptoanalyse) = Wissenschaft von der Entzifferung von Geheimtexten
 - *Kryptographie* = Wissenschaft von den Verschlüsselungstechniken = dem Erzeugen gut verschlüsselter Texte
- **Schlüssel** (= Passwort)
- **Zielvorgabe:**
 - Verschlüsselung und Entschlüsselung muss (für einen Computer) einfach sein, wenn der Schlüssel bekannt ist.
 - Ohne Kenntnis des Schlüssels soll eine Enträtselung / Entschlüsselung nicht möglich sein = selbst mit beträchtlichen Ressourcen oder in Kenntnis des Verfahrens.

Konsequenz: im obigen Beispiel ist

1. `xvd-sulydwh-gdwhq-yrq-vlfkhukhlwvsrolwlnhuq-lp-qhwC` das Chiffre,
2. das mit dem Caesar-Verfahren *entschlüsselt* wird,
3. wenn man die Zahl 3 als Schlüssel hat.
4. Wenn man den Schlüssel nicht hat, muss man den verschlüsselten String (das Chiffre) *entziffern* (dechiffrieren).
5. Die Zielvorgabe ist nicht erfüllt: Entschlüsselung ist einfach, Entzifferung (ohne Schlüssel) aber auch (Brute-Force-Ausprobieren.) (hängt linear von der Anzahl der Buchstaben im Alphabet ab.)

ÜBUNG LF09/16:Verschlüsselung:02

Vorbereitung:

- ☐ Vereinbaren Sie einen Kanal/Medium, über den/das Sie Nachrichten an alle Mitglieder der Klasse versenden können.
- ☐ Teilen Sie sich in zwei Gruppen auf.

Ausarbeitung:

- ☐ Einigen Sie sich in jeder Gruppe auf einen Verschlüsselungsalgorithmus und einen Schlüssel. Lassen Sie Ihre Fantasie walten.
- ☐ Testen Sie Prozedere in der Gruppe.
- ☐ Jede Gruppe sende eine 'Emissärin' aus Ihrer Gruppe mit ihrem Messenger zur Lehrerin.
- ☐ Jede 'Emissärin' setze sich deutlich getrennt von Ihrer Gruppe (und der Konkurrenz) hin und
 - ☐ empfangen eine spezifische Nachricht von der Lehrerin,
 - ☐ verschlüssele diese Nachricht gemäß des Gruppenalgorithmus / Schlüssels
 - ☐ versende diese Nachricht über den für alle Schülerinnen zugänglichen Kanal.
- ☐ Jede Gruppe
 - ☐ empfangen die Nachricht,
 - ☐ entschlüssele sie,
 - ☐ notiere sie still auf einem Zettel
 - ☐ übergebe den an die Lehrerin,
 - ☐ entziffere die verschlüsselte Nachricht der anderen Gruppe,
 - ☐ dokumentieren das eigene Verschlüsselungsverfahren,
 - ☐ und skizziere das Verschlüsselungsverfahren der Gegenseite

Wer die Nachricht der anderen Gruppe zuerst entziffert hat, hat gewonnen.

Resumée:

- ☐ Jede Gruppe gebe die Nachricht der anderen bekannt,
- ☐ Beschreibe das Verschlüsselungsverfahren der Gegenseite,
- ☐ Beschreibe das eigene Verschlüsselungsverfahren.

Hinweis:

- Sie dürfen für die Verschlüsselung, Entschlüsselung und/oder Entzifferung jede Methode anwenden, die Ihnen einfällt - außer solche, die Gewalt beinhalten.
- Überlegen Sie schon bei der Konzeption Ihrer Verschlüsselung ein Verfahren zur Entzifferung / Entschlüsselung der Gegenseite.

Lösung:

- **1.:** Matts, Daniel (GSLDK, 11IV24):

- wähle ein Pangramm = "The quick brown fox jumps over the lazy dog" als Schlüssel,
 - weise jedem Buchstaben des Alphabets eine der Nummern zu, die er auch im Pangramm hat (T=0, h=1, ...)
 - ersetze die Buchstaben im Satz 'Tief ist der Brunnen der Vergangenheit' durch diese Nummern, das Blank bleibe eine Blank
 - → 0,5,2,13 5,20,25 32,23,9 8,24,4,12,12,27,12 2,23,9 22,2,9,34,29,12,34,2,12,1,2,5,25
- **2.:** Waldemar, David, Faezeh, Juliana, Evelina, Alexander, Dominika (GSLDK, 11IV24):
 - lege in einer Substitutionstabelle fest, welcher Buchstabe welche Zahl haben soll u.u.
 - ersetze die Buchstaben im Satz 'Morgen war Weihnachten' durch diese Nummern, das Blank habe die Nummer 00
 - → 08 02 25 26 01 15 00 16 17 25 00 16 01 19 06 15 17 22 06 13 01 15
-

3. Verfahren: [→ ZP:Sheet3]

- **Symmetrische Verfahren** verwenden zur Ver- und Entschlüsselung denselben Schlüssel
 - *Substitutionsverfahren* = z.B. Monoalphabetische Substitution: Caesar-Verschlüsselung [A>C]
 - *Transpositionsverfahren* = Durcheinanderwürfeln des Textes
 - *Stromverschlüsselung*: die Zeichen des Klartextes jeweils einzeln und nacheinander verschlüsselt.
 - *Blockverschlüsselung*: Klartext vorab in Blöcke aufgeteilt, die (kontextbedingt) verschlüsselt werden
 - moderne Verfahren:
 - * **RC4** (Rivest Cipher 4)
 - verarbeitet Daten als Bitstrom
 - Schwachstellen bekannt
 - * **AES** (Advanced Encryption Standard)
 - = 1998 entwickelt
 - Schlüssellänge von 128, 192 oder 256 Bit
 - arbeitet mit festen 128-Bit-Datenblöcke
 - gilt als sicher
 - Vorteil: schnellere Berechnung

- Nachteil: Schlüssel ist mehreren bekannt.
- **Asymmetrische Verfahren** (= Public-key cryptography), benutzen zur Verschlüsselung einen “völlig anderen” Schlüssel als zur Entschlüsselung
 - *öffentlicher Schlüssel* zum Verschlüsseln
 - * wird weitergegeben
 - * kann den Text NICHT entschlüsseln, (nicht mal vom Verschlüssler)
 - *privater Schlüssel* zum Entschlüsseln
 - * wird NICHT weitergegeben
 - *Bedingung*
 - * privater Schlüssel darf nicht in absehbarer Zeit (Jahre) aus dem öffentlichen Schlüssel berechnet werden können
 - * benötigt **Einwegfunktion** = eine mathematische Funktion, die einfach zu berechnen ist, ihre Umkehrung jedoch nur sehr schwer.
 - Modulo-Division
 - einfach zwei große Primzahlen zu multiplizieren, jedoch sehr schwer das Ergebnis wieder in seine Primfaktoren zu zerlegen.
 - Beispiel
 - * 1975: erste Idee von Diffie und Hellmann
 - * 1977: **RSA**=Algorithmus = erste anwendbarer Funktion
 - erfunden L. Rivest, A. Shamir und L. M. Adleman
 - Grundidee: es gibt keinen effizienten Algorithmus gibt, um eine große natürliche Zahl in ihre Primfaktoren zu zerlegen.
 - * 1978 McEliece Kryptosystem,
 - * 1979 das Rabin Kryptosystem,
 - * 1984 das Chor-Rivest Kryptosystem u
 - * 1985 das ElGamal Kryptosystem
 - Vorteil: Schlüsselaustausch über unsichere Netzwerke möglich.
 - Nachteil: die erhöhte Rechenleistung (RSA 1000 mal langsamer als AES)
- **hybride Verfahren**: organisieren Schlüsselaustausch asymmetrisch, die Verschlüsselung symmetrisch.

[→ ZP:Sheet4]

ÜBUNG LF09/16:Verschlüsselung:03

- ☐ Bitte entziffern Sie den folgenden String. (Der Einfachheit halber auch als Hex-Valuestream)

```
1 ?\x1e\t[/\x1e\x0e\x1d\x1e\x17[\x0f\t\xb8\xdf\x1c\x0f[+\t\x1a\x1f\x1a'  
2  
3 =  
4  
5 0x3f0x1e0x90x5b0x2f0x1e0xe0x1d0x1e0x170x5b  
6 0xf0x90xb80xdf0x1c0xf0x5b0x2b0x90x1a0x1f0x1a
```

- ☐ Beschreiben Sie den Algorithmus

Sie erhalten - wenn nötig – in kürzeren Abständen kleine Hilfestellungen.

Hilfen:

1. Der Algorithmus lässt sich ohne jede Programmänderung zum Verschlüsseln und Entschlüsseln verwenden.
2. Es geht um einen gerade aktuellen Film.
3. Der Schlüssel ist 123.
4. XOR-Verschlüsselungsmethodik [**→ ZP:Sheet5**]

Lösung:

- Der Teufel trägt Prada bytewise XOR-verknüpft mit Schlüssel 123 ergibt

```
1 ?\x1e\t[/\x1e\x0e\x1d\x1e\x17[\x0f\t\xb8\xdf\x1c\x0f[+\t\x1a\x1f\x1a'
```

- der String wieder bytewise XOR-verknüpft mit Schlüssel 123 ergibt Der Teufel trägt Prada

[**→ ZP:Sheet6**] mit Vorführung in VSCODE

Seitenfrage: Wo wird diese XOR-Fähigkeit noch genutzt?

Seitenantwort: Exkurs zu RAID (s [lf.cx/cx.raid*](#))?

5. Hashverfahren [**→ ZP:Sheet7**]

Eine **Hashfunktion** bildet einen Text variabler Länge (große Daten) auf kleineren Text/Zahl fester Länge ab:

- Der Hashwert sollte für jeden Text eindeutig sein.

- Kurzer Hash:
 - Vorteil: schnellere Berechnung
 - Nachteil: größere Gefahr der Kollision.
- Langer Hash:
 - aufwendige Berechnung
 - kleine Kollisionsgefahr.
- **MD5** (= Message Digest 5) älterer Algorithmus v. 1991.
- **SHA** (Secure Hash Algorithm) in verschiedenen Versionen
 - SHA-1 = 160-Bit-Ausgabe
 - SHA-2 = 224-, 256-, 384- oder 512-Bit-Ausgabe
 - SHA-3 = 24-, 256-, 384- oder 512-Bit-Ausgabe
 - SHA256 spezielle Version von SHA-2
- Entscheidungskriterium
 - ‘sicher’: SHA-2 oder SHA-3
 - Effizienz: MD5 oder SHA-1 (aber knackbar = leicht Kollision zu erzeugen)
- Anwendung:
 - Download-Verifikation
 - s. google.
- Anwendungen
 - bash/LX: `echo "Hello world" | openssl sha256`
 - bash/LX: `echo 'hello world' | sha256sum`
 - powershell/WIN: `Get-FileHash`
 - commandshell: `CertUtil`

6. Signatur/Unterschrift: Nachweise der Autorenschaft [→ ZP:Sheet7]

- Allgemeines Eigenschaft: *öffentliche Schlüssel* können entschlüsseln, was *Private Schlüssel* **verschlüsselt** haben. (Reziproke Anwendung)
- Anwendung:
 - Aus Text wird ein Hash erzeugt.
 - Autor verschlüsselt des Hash mit seinem privaten Schlüssel.
 - reicht beides zusammen weiter.

- Empfänger des Textes entschlüsselt Text mit öffentlichem Schlüssel des Autors: Nachweis erbracht.
- Voraussetzung: verlässliche Schlüsselweitergabe **Schlüsselparties**

7. Zwischenfazit

A: Kommunikation ohne Authentifizierung

- Sender:
 - organisiert sich den öffentlichen Schlüssel des Empfängers,
 - **verschlüsselt** den Text mit dem öffentlichen Schlüssel des Empfängers,
 - sendet die verschlüsselte Nachricht an Empfänger.
 - Die **verschlüsselte Nachricht kann jetzt nur noch vom Besitzer des privaten Schlüssels dekodiert werden.
 - **ABER:** (Ohne Zusatzmaßnahmen) weiß der Sender nicht, wer der Empfänger / Besitzer des privaten Schlüssels *in persona* ist.
- Empfänger:
 - empfängt eine Nachricht,
 - entschlüsselt diese mit seinem privaten Schlüssel.
 - Der Empfänger kennt die Nachricht, weiß aber nicht wirklich, von wem sie kommt. (Mailabsender könnte gehackt sein)

Lösung (I)

Man gibt seine public-Keys auf Schlüsselparties in persona weiter. Dann

- weiß der Sender, dass der Empfänger der intendierte Adressat ist:
 - den öffentlichen Schlüssel hat der Sender vom intendierten Empfänger bekommen,
 - nur der Besitzer des *Private Keys* kann die Nachricht lesen
 - ist die Adresse des intendierten Empfängers korrumpiert, bleibt der Inhalt geschützt
- Problem 1: Wenn der ganze Account des Empfängers samt Schlüsseln gehackt, ist kein Schutz mehr da
- Problem 2: Der Empfänger weiß nicht, von wem die Nachricht wirklich kommt.

Lösung (II):

Zusätzlich zu (I)

- Sender:

- erzeugt einen Hashwert der (unverschlüsselten) Nachricht,
- verschlüsselt den Hashwert mit seinem **privaten Key**
- sendet beides an den Empfänger.
- Empfänger:
 - entschlüsselt die Nachricht mit seinem privaten Key (und kennt den Inhalt)
 - bildet den Empfänger-Hashwert aus der (unverschlüsselten) Nachricht
 - entschlüsselt den verschlüsselten Hash-Wert mit dem öffentlichen Schlüssel des Senders,
 - vergleicht den Empfänger-Hashwert mit dem Empfänger-Hashwert: sind beide gleich, weiß der, dass der Sender in persona der ist, der er zu sein vorgibt.

ABER: Diese Verfahren funktioniert nur, wenn beide Parteien ihre öffentlichen Schlüssel NUR in persona austauschen (und sie sich nicht ersatzweise einfach so herunterladen!)

Konsequenz:

Wir wissen bereits, dass es verschlüsselte Kommunikationen ohne Schlüsselparties (persönliche Authentifizierung) gibt.

ERGO: Es muss andere Verfahren geben:

8. Verschlüsselung in Netzwerken

- **Leitungsverschlüsselung** = Nachricht nur jeweils für den Nachbarrechner verschlüsselt, der entschlüsselt die Nachricht, verschlüsselt sie wiederum (mit einem möglicherweise anderen Verfahren) und schickt sie an seinen Nachbarn
 - Vorteil: nur Nachbarrechner müssen sich auf ein Verschlüsselungsverfahren und verwendete Schlüssel einigen
 - Vorteil: Übertragung auf niedriger Protokollebene (auch Übertragungs-Hardware) möglich
 - Nachteil: jeder einzelne Rechner auf dem Übertragungsweg muss vertrauenswürdig und sicher sein.
- **Ende-zu-Ende-Verschlüsselung** = Nachricht vom Absender verschlüsselt und in dieser Form unverändert über mehrere Rechner hinweg zum Empfänger übertragen
 - Vorteil: Man-in-the-Middle-Attacks nicht möglich
 - Nachteil: Sender muss mit jedem Empfänger ein Verschlüsselungsverfahren und zugehörige(n) Schlüssel ausmachen

[→ **ZP:Sheet78**]

- **TSL** (= Transport Layer Security) Verschlüsselung zwischen Layer VII und Layer IV

- Beispiels <https> - aber auch andere.
 - TLS 1.0/1999 1.1/2006 1.2/2008 1.3/2018
 - Vorgänger: Secure Sockets Layer (SSL 1.0/1994 - SSL 3.0/1996)
 - **IPSEC** (Internet Protocol Security)
 - arbeitet direkt auf der Vermittlungsschicht (Internet Layer, entspricht OSI Layer 3)
 - soll verschlüsselungsbasierte Sicherheit auf Netzwerkebene bereitstellen
 - soll das Mitlesen beim Umschreiben / Routing verhindern
 - führt bei NAT u.U. zu Problem (Änderung quell-IP)
-

ÜBUNG LF09/16:Verschlüsselung:04

- ☐ Bitte ermitteln und beschreiben Sie die Struktur eines Zertifikats.
- ☐ Bitte 'erfinden' Sie (ohne die Hilfe von ChatGPT, Wikipedia etc.) ein Verfahren/Prozess/Protokoll, bei dem die Kommunikation zwischen Client und Server
 - ☐ automatisiert 'on the fly' = adhoc verschlüsselt wird, also
 - ☐ ohne den Austausch vorab authentifizierter Schlüssel auskommt
 - ☐ und trotzdem abgesichert ist
- ☐ Bitte ermitteln Sie die prinzipiellen Grenzen Ihres Verfahrens/Prozesses/Protokolls.
- ☐ Bitte ermitteln Sie danach den Unterschied zwischen Ihrem Verfahren und TLS (bzw. IPSEC)

Lösung: [→ **ZP:Sheet8-10**]

ÜBUNG LF09/16:Verschlüsselung:05

- ☐ Bitte beschreiben Sie, wie eine 2-Faktorauthentifizierung die Grenzen Ihres und des TLS-Verfahrens absichern kann.
- ☐ Bitte beschreiben Sie, wie das z.B. beim Internet-Banking eingesetzt wird.

Lösung:

- Kunde hinterlegt den Zugang zu einem vom ersten Kommunikationsweg unabhängigen Mittel zum Informationsaustausch.
 - Smartphonenummer für Zusendung von SMS-Freigabecode
 - Verifikationsapp auf einem Mobile-Phone

- Einmal-Tan-Generator / Einmal-Tan-Übersendungsapp (à la GitHub).
 - → Vorführung GitHub-Zugriff mit eigenem GH-Account mit 2-Faktorauthentifizierung per [Aegis](#)
-

9. Hinweise

- → <https://de.wikipedia.org/wiki/Verschl%C3%A4sslung>
- → [BSI/Datenverschlüsselung](#)
- → <https://studyflix.de/informatik/asymmetrische-verschlusselung-1609>
- → <https://www.conductor.com/de/academy/glossar/ssl-verschluesselung/>
- → <https://www.cloudflare.com/de-de/learning/ssl/transport-layer-security-tls/>
- → <https://www.cloudflare.com/de-de/learning/ssl/transport-layer-security-tls/>
- → <https://www.ibm.com/docs/de/ibm-mq/9.2?topic=tls-overview-ssltls-handshake>
- https://de.wikipedia.org/wiki/Transport_Layer_Security
- → <https://www.ecos.de/blog/asymmetrische-verschluesselung-einfach-erklaert>
- → <https://www.heise.de/tipps-tricks/SSH-Tunnel-nutzen-so-geht-s-4320041.html>
- → https://de.wikipedia.org/wiki/Virtual_Private_Network
- → <https://aws.amazon.com/de/what-is/vpn/>
- → <https://de.wikipedia.org/wiki/IPsec>
- → <https://www.security-insider.de/was-ist-ipsec-a-781354/>